

Download Economic & Financial Data from Institutional Data Providers

- Part 2 - BIS - Bank for International Settlements -

If you don't have a paid data provider and you're looking for core economic statistics, you can rely on institutional sources to build your repository, including:

- [Bank for International Settlements -BIS-](#)
- [Federal Reserve Bank of St. Louis -FRED-](#) - ⚠️ You'll need a free FRED API key that you can get here: [FRED API](#)
- [European Central Bank -ECB-](#)
- [EUROSTAT](#)
- [Banca d'Italia -BANKIT-](#)

In this notebook I show how to retrieve data from the [Bank for International Settlements -BIS-](#)

I will use SDMX RESTful API web service that offers programmatic access to data and metadata . It provides a standardised interface for interacting with software systems implementing the SDMX standard.[SDMX standard:https://github.com/sdmx-twg/sdmx-rest/](https://github.com/sdmx-twg/sdmx-rest/)

Refer to the official [BIS SDMX RESTful API](#) for full specifications.

I demonstrate how to use the API and implement a set of utility functions to interact with the Statistical Database, along with several examples.

```
In [2]: #install
import subprocess
import sys
# If needed, install dependencies (uncomment to run)
def install_packages():
    packages = ['lxml', 'pandasdmx', 'pandasql', 'matplotlib', 'seaborn', 'scikit-learn', 'statsmodels', 'scipy', 'requests',
               ]
    for package in packages:
```

```

        try:
            subprocess.check_call([sys.executable, "-m", "pip", "install", package])
        except:
            print(f"Package {package} already installed or failed to install")
install_packages()
#####
#Standard Library
import math
import gzip
import zipfile
import io
import os
import shutil
import subprocess
import sys
import time
from datetime import datetime, timedelta
import warnings
from functools import reduce
from io import StringIO
import tkinter as tk
from tkinter import Tk, filedialog
from tkinter.filedialog import askopenfilename
from IPython import display
from IPython.display import display # Import the standard display function

# Other Library Imports
import re
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pandas_datareader.data as web
from pandasql import sqldf
import requests
import xml.etree.ElementTree as ET
import seaborn as sns
import statsmodels.api as sm
from scipy import stats
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

```

```

from sklearn.preprocessing import StandardScaler
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.tsa.api import VAR, VECM
from statsmodels.tsa.stattools import adfuller, coint
from pathlib import PurePosixPath

from fredapi import Fred
import eurostat

# Define the SQL runner
pysqldf_G = lambda q: sqldf(q, globals())
pysqldf_L = lambda q: sqldf(q, locals())
warnings.filterwarnings("ignore")

#####
#Constants
EUROSTAT3_URL = "https://ec.europa.eu/eurostat/api/dissemination/sdmx/3.0/data"
EUROSTAT2_URL = "https://ec.europa.eu/eurostat/api/dissemination/sdmx/2.1/data"
ECB_SDW = "https://sdw-wsrest.ecb.europa.eu/service/data"
ECB_BULK_DATA = "https://data-api.ecb.europa.eu/service/data"
BANKIT_URL = "https://a2a.bancaditalia.it/infostat/dataservices/export"
DT_START = '2001-12-31'
DT_END = '2024-12-31'

```

Package jsonstat.py already installed or failed to install

BIS Statistical Database

[Here the BIS API Official user guide: bis api doc.yaml](#)

- We define some utility function and global variables

```

In [3]: from requests.adapters import HTTPAdapter
        from urllib3.util.retry import Retry

        def make_session(retries=3, backoff=0.5, status_forcelist=(429, 500, 502, 503, 504)):
            s = requests.Session()

```

```

retry = Retry(
    total=retries, read=retries, connect=retries,
    backoff_factor=backoff, status_forcelist=status_forcelist,
    allowed_methods=frozenset(["GET"]),
    raise_on_status=False,
)
s.mount("https://", HTTPAdapter(max_retries=retry))
s.mount("http://", HTTPAdapter(max_retries=retry))
return s

```

#Create Global Variables

```
BASE = "https://stats.bis.org/api/v2"
```

```
SESSION = make_session()
```

```
DEFAULT_TIMEOUT = 180
```

```

In [4]: def list_flows(sess=requests.session=SESSION, timeout:int = DEFAULT_TIMEOUT):
        """
        Return a DataFrame with BIS dataflows (flow ID + Description).
        """
        url = f"{BASE}/structure/dataflow/BIS/all"
        # Ask for SDMX-Structure XML (widely supported). JSON also works at many SDMX endpoints.
        hdr = {"Accept": "application/vnd.sdmx.structure+xml;version=2.1"}
        sess = make_session() if sess is None else sess
        r = sess.get(url, headers=hdr, timeout=timeout)
        r.raise_for_status()
        ns = {
            "mes": "http://www.sdmx.org/resources/sdmxml/schemas/v2_1/message",
            "str": "http://www.sdmx.org/resources/sdmxml/schemas/v2_1/structure",
            "com": "http://www.sdmx.org/resources/sdmxml/schemas/v2_1/common",
            "xml": "http://www.w3.org/XML/1998/namespace",
        }
        root = ET.fromstring(r.content)
        rows = []
        for df in root.findall("./str:Dataflow", ns):
            flow_id = df.attrib.get("id")
            # pick English name if present
            label = ""
            for nm in df.findall("com:Name", ns):
                if nm.attrib.get("{%s}lang" % ns["xml"], "").lower() == "en":
                    label = (nm.text or "").strip(); break

```

```

        if not label:
            nm = df.find("com:Name", ns)
            label = (nm.text or "").strip() if nm is not None else ""
        if flow_id:
            rows.append({"flow_id": flow_id, "Description": label})
    df = pd.DataFrame(rows).sort_values("flow_id").reset_index(drop=True)
    return df

flows = list_flows()
flows.head(10)

```

Out[4]:

	flow_id	Description
0	BIS_REL_CAL	BIS_RELEASE_CALENDAR
1	WS_CBPOL	Central bank policy rates
2	WS_CBS_PUB	BIS consolidated banking
3	WS_CBTA	Central bank total assets
4	WS_CPMI_CASHLESS	CPMI cashless payments (T5,T6)
5	WS_CPMI_CT1	CPMI comparative tables type 1
6	WS_CPMI_CT2	CPMI comparative tables type 2
7	WS_CPMI_DEVICES	CPMI payment devices (T4)
8	WS_CPMI_INSTITUT	CPMI institutions (T3)
9	WS_CPMI_MACRO	CPMI macro (T1,T2)

bis_flow_ids: download all the flow id list

```

In [5]: def bis_flow_ids(sess:requests.session=None, timeout=DEFAULT_TIMEOUT) -> pd.DataFrame: #get the complete flow list
        # Try a few SDMX-structure endpoints (v2 first, then v1)
        endpoints = [
            "https://stats.bis.org/api/v2/structure/dataflow/BIS/all",
            "https://stats.bis.org/api/v2/structure/dataflow/BIS",
            "https://stats.bis.org/api/v2/structure/dataflow",

```

```

        "https://stats.bis.org/api/v1/dataflow/BIS/all",
        "https://stats.bis.org/api/v1/dataflow/BIS",
        "https://stats.bis.org/api/v1/dataflow",
    ]
    hdr = {"Accept": "application/vnd.sdmx.structure+xml;version=2.1"}
    ns = {
        "mes": "http://www.sdmx.org/resources/sdmxml/schemas/v2_1/message",
        "str": "http://www.sdmx.org/resources/sdmxml/schemas/v2_1/structure",
        "com": "http://www.sdmx.org/resources/sdmxml/schemas/v2_1/common",
        "xml": "http://www.w3.org/XML/1998/namespace",
    }
    sess = make_session() if sess is None else sess
    last_err = None
    for url in endpoints:
        try:
            r = sess.get(url, headers=hdr, timeout=timeout)
            r.raise_for_status()
            root = ET.fromstring(r.content)
            rows = []
            for df in root.findall("./str:Dataflow", ns):
                # keep only BIS-maintained flows if agencyID is set
                agency = df.attrib.get("agencyID")
                if agency and agency != "BIS":
                    continue
                fid = df.attrib.get("id")
                # prefer English name
                name = None
                for nm in df.findall("com:Name", ns):
                    if nm.attrib.get(f"{{{ns['xml']}}}lang", "").lower() == "en":
                        name = (nm.text or "").strip()
                        break
                if name is None:
                    nm = df.find("com:Name", ns)
                    name = (nm.text or "").strip() if nm is not None else ""
                if fid:
                    rows.append({"flow_id": fid, "Description": name})
            if rows:
                return (pd.DataFrame(rows)
                        .drop_duplicates("flow_id")
                        .sort_values("flow_id")
                        .reset_index(drop=True))

```

```

        except Exception as e:
            last_err = e
            continue
        raise RuntimeError(f"Could not retrieve BIS dataflows. Last error: {last_err}")
#####
# Example:
flows = bis_flow_ids(SESSION)
display(flows.head())
print('#'*120)
display(flows[flows.flow_id=="WS_CREDIT_GAP"])

```

	flow_id	Description
0	BIS_REL_CAL	BIS_RELEASE_CALENDAR
1	WS_CBPOL	Central bank policy rates
2	WS_CBS_PUB	BIS consolidated banking
3	WS_CBTA	Central bank total assets
4	WS_CPMI_CASHLESS	CPMI cashless payments (T5,T6)

#####

	flow_id	Description
13	WS_CREDIT_GAP	BIS credit-to-GDP gaps

- **load_flow**: download a single flow_id

bis_series_keys: download all the series keys for a given flow id

```

In [6]: def list_series_keys_base(flow_id, where="all",key_path: str = "", version: str = "1.0"):
        """
        Use SDMX 'serieskeyonly' to get valid series keys for a flow.
        Returns a DataFrame with at least a 'KEY' (or similar) column.
        """
        base = "https://stats.bis.org/api/v2/data/dataflow/BIS"
        # When key_path is empty, most BIS flows accept the trailing slash.

```

```

url = f"{base}/{flow_id}/{version}/{key_path}".rstrip("/") + "?format=csv&detail=serieskeyonly"
# Use the SESSION object with retries defined in cell g41cXaPIzpBE
r = SESSION.get(url, timeout=DEFAULT_TIMEOUT)
r.raise_for_status()
df = pd.read_csv(io.StringIO(r.text), dtype=str, low_memory=False)
return df

```

```
#####
```

```
#Example:
```

```
flow_id = "WS_CREDIT_GAP"
```

```
keys = list_series_keys_base(flow_id=flow_id)
```

```
display(keys.head())
```

	FREQ	BORROWERS_CTY	TC_BORROWERS	TC_LENDERS	CG_DTYPE
0	Q	AU	P	A	A
1	Q	MY	P	A	A
2	Q	NL	P	A	C
3	Q	NL	P	A	B
4	Q	ZA	P	A	C

```

In [8]: def list_series_keys(flow_id, sess=requests.session = SESSION, where="all",key_path: str = "",
                             version: str = "1.0" ):
    """
    Use SDMX 'serieskeyonly' to get valid series keys for a flow.
    Returns a DataFrame with at least a 'KEY' (or similar) column.
    """
    base = "https://stats.bis.org/api/v2/data/dataflow/BIS"
    # When key_path is empty, most BIS flows accept the trailing slash.
    url = f"{base}/{flow_id}/{version}/{key_path}".rstrip("/") + "?format=csv&detail=serieskeyonly"
    # Use the SESSION object with retries defined in cell g41cXaPIzpBE
    sess = make_session() if sess is None else sess
    r = sess.get(url, timeout=DEFAULT_TIMEOUT)
    r.raise_for_status()
    df = pd.read_csv(io.StringIO(r.text), dtype=str, low_memory=False)
    df['INDEX_KEY'] = df[df.columns].agg(".".join, axis=1)

```



```

df['KEY']=df['INDEX_KEY']
df.set_index('INDEX_KEY',inplace=True)
df = df[['KEY'] + [c for c in df.columns if c != 'KEY']]
return df
#####
# Example:
flow_id = "WS_CREDIT_GAP"
df_keys = list_series_keys(flow_id=flow_id)
display(df_keys.head())

```

	KEY	FREQ	BORROWERS_CTY	TC_BORROWERS	TC_LENDERS	CG_DTYPE
INDEX_KEY						
	Q.AU.P.A.A	Q	AU	P	A	A
	Q.MY.P.A.A	Q	MY	P	A	A
	Q.NL.P.A.C	Q	NL	P	A	C
	Q.NL.P.A.B	Q	NL	P	A	B
	Q.ZA.P.A.C	Q	ZA	P	A	C

```

In [9]: #Raw and shorter version
def bis_series_keys(flow_id: str, key_path: str = "", version: str = "1.0") -> pd.DataFrame:
    """
    Return the series keys for a BIS dataflow (no observations).
    - flow_id: e.g. 'WS_CREDIT_GAP', 'WS_CBTA'
    - key_path: the dot-separated SDMX key. Leave '' to request "all".
      (You can also narrow e.g. 'Q..B..' etc depending on the flow.)
    """
    base = "https://stats.bis.org/api/v2/data/dataflow/BIS"
    # When key_path is empty, most BIS flows accept the trailing slash.
    url = f"{base}/{flow_id}/{version}/{key_path}".rstrip("/") + "?format=csv&detail=serieskeyonly"
    df = pd.read_csv(url, dtype=str)
    df['INDEX_KEY'] = df[df.columns].agg(".", join, axis=1)
    df['KEY']=df['INDEX_KEY']
    df.set_index('INDEX_KEY',inplace=True)
    df = df[['KEY'] + [c for c in df.columns if c != 'KEY']]

```

```
return df
```

```
#####
```

```
#Example:
```

```
flow_id = "WS_CREDIT_GAP"
```

```
df_keys = bis_series_keys(flow_id)
```

```
display(df_keys)
```

	KEY	FREQ	BORROWERS_CTY	TC_BORROWERS	TC_LENDERS	CG_DTYPE
INDEX_KEY						
	Q.AU.P.A.A	Q	AU	P	A	A
	Q.MY.P.A.A	Q	MY	P	A	A
	Q.NL.P.A.C	Q	NL	P	A	C
	Q.NL.P.A.B	Q	NL	P	A	B
	Q.ZA.P.A.C	Q	ZA	P	A	C
	...	...	...	...	...	...
	Q.FI.P.A.C	Q	FI	P	A	C
	Q.FI.P.A.B	Q	FI	P	A	B
	Q.IL.P.A.C	Q	IL	P	A	C
	Q.IL.P.A.B	Q	IL	P	A	B
	Q.IE.P.A.A	Q	IE	P	A	A

132 rows × 6 columns

- **get_series**: download a single series using the BIS SDMX REST API

```
In [10]: def get_series(flow_id, series_key, sess: requests.Session=SESSION, start=None, end=None):
        """
        Download a single series as a tidy DataFrame (one observation per row).
```

here the structure of the url:

../data/{context}/{agencyID}/{resourceID}/{version}/{key}:

- context: e.g. 'dataflow'
 - agencyID: e.g. 'BIS'
 - resourceID: e.g. 'WS_CREDIT_GAP', 'WS_CBTA'
 - version: e.g. '1.0'
 - series_key: e.g. 'Q.IT.P.A.A'
- """

#/data/{context}/{agencyID}/{resourceID}/{version}/{key}:

url = f"{BASE}/data/dataflow/BIS/{flow_id}/1.0/{series_key}?"

params = {

 "format": "csv", *# CSV is easy to read with pandas*

 "detail": "dataonly", *# no redundant attributes*

}

if start: params["startPeriod"] = start

if end: params["endPeriod"] = end

sess= make_session() if sess is None else sess

r = sess.get(url, params=params,timeout=DEFAULT_TIMEOUT)

r.raise_for_status()

df = pd.read_csv(io.StringIO(r.text), dtype=str, low_memory=False)

Standard SDMX-CSV has OBS_VALUE / TIME_PERIOD columns; coerce numeric values if present

if "OBS_VALUE" in df.columns:

 df["OBS_VALUE"] = pd.to_numeric(df["OBS_VALUE"], errors="coerce")

return df

#####

#Example:

series_key = "Q.IT.P.A.A"

df_series = get_series(flow_id, series_key)

display(df_series.head())

	FREQ	BORROWERS_CTY	TC_BORROWERS	TC_LENDERS	CG_DTYPE	TIME_PERIOD	OBS_VALUE
0	Q	IT	P	A	A	1960-Q4	53.5
1	Q	IT	P	A	A	1961-Q1	54.9
2	Q	IT	P	A	A	1961-Q2	56.1
3	Q	IT	P	A	A	1961-Q3	57.1
4	Q	IT	P	A	A	1961-Q4	57.8

Bulk downloads

BIS publishes ready-made ZIPs (CSV or SDMX) for each topic on the Bulk downloads page. Grab the file you need and read the CSV.

Tips:

- The bulk page lists every topic (policy rates, credit, property prices, etc.) with links like [.../static/bulk/WS__csv_flat.zip](#).

Portale Dati BIS

- You can also reach bulk links from any topic's "Export → Bulk downloads" dialog.

```
In [11]: def load_bis_bulk(flow_id:str, sess:requests.session = SESSION, prefer_flat=True, timeout=DEFAULT_TIMEOUT) -> pd.DataFrame:
    CANDIDATES = [
        # current canonical paths
        f"https://data.bis.org/static/bulk/{flow_id}_csv_flat.zip",
        f"https://data.bis.org/static/bulk/{flow_id}_csv_col.zip",
        # fallbacks (older path sometimes works)
        f"https://data.bis.org/bulkdownload/{flow_id}_csv_flat.zip",
        f"https://data.bis.org/bulkdownload/{flow_id}_csv_col.zip",
    ]

    urls = CANDIDATES if prefer_flat else CANDIDATES[1:2] + CANDIDATES[0:1] + CANDIDATES[2:]
    last_err = None
    sess = make_session() if sess is None else sess
    for url in urls:
```

```

try:
    r = sess.get(url, timeout=timeout)
    r.raise_for_status()
    with zipfile.ZipFile(io.BytesIO(r.content)) as z:
        # pick the first CSV inside the zip
        name = [n for n in z.namelist() if n.lower().endswith(".csv")][0]
        df = pd.read_csv(z.open(name), dtype=str)
        return df
except Exception as e:
    last_err = e
    continue
raise RuntimeError(f"All download attempts failed. Last error: {last_err}")

#####
# Example:
flow_id= "WS_CREDIT_GAP"
df_bis_Flat = load_bis_bulk(flow_id,prefer_flat=True)
df_bis_NoFlat = load_bis_bulk(flow_id,prefer_flat=False)
display(df_bis_Flat.head())
print('#'*150)
display(df_bis_NoFlat.head())

```

STRUCTURE		STRUCTURE_ID	ACTION	FREQ:Frequency	BORROWERS_CTY:Borrowers' country	TC_BORROWERS:Borrowing sector	TC_LENDERS:Lenders
0	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	AU: Australia	P: Private non-financial sector	A: All sectors
1	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	AU: Australia	P: Private non-financial sector	A: All sectors
2	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	AU: Australia	P: Private non-financial sector	A: All sectors
3	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	AU: Australia	P: Private non-financial sector	A: All sectors
4	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	AU: Australia	P: Private non-financial sector	A: All sectors

#####

#####

	FREQ	Frequency	BORROWERS_CTY	Borrowers' country	TC_BORROWERS	Borrowing sector	TC_LENDERS	Lending sector	CG_DTYPE	Credit gap data type	...	2022-Q3	2021-Q3
0	Q	Quarterly	AU	Australia	P	Private non-financial sector	A	All sectors	A	Credit-to-GDP ratios (actual data)	...	174.5	174.5
1	Q	Quarterly	MY	Malaysia	P	Private non-financial sector	A	All sectors	A	Credit-to-GDP ratios (actual data)	...	156.3	156.3
2	Q	Quarterly	NL	Netherlands	P	Private non-financial sector	A	All sectors	C	Credit-to-GDP gaps (actual-trend)	...	-40.2	-40.2
3	Q	Quarterly	NL	Netherlands	P	Private non-financial sector	A	All sectors	B	Credit-to-GDP trend (HP filter)	...	350.4	350.4
4	Q	Quarterly	ZA	South Africa	P	Private non-financial sector	A	All sectors	C	Credit-to-GDP gaps (actual-trend)	...	-7.8	-7.8

5 rows × 323 columns



```
In [12]: import io, zipfile, requests, pandas as pd
         from urllib.parse import urljoin
```

```

from bs4 import BeautifulSoup

BULK_PAGE = "https://data.bis.org/bulkdownload"

def list_bulk_csv_flat_links(timeout:int=120) -> list[str]:
    """Scrape the BIS Bulk downloads page for all *_csv_flat.zip files."""
    html = requests.get(BULK_PAGE, timeout=timeout).text
    soup = BeautifulSoup(html, "lxml")
    links = []
    for a in soup.find_all("a", href=True):
        href = a["href"]
        if href.endswith("_csv_flat.zip"):
            links.append(urljoin("https://data.bis.org", href))
    return sorted(set(links))

def parse_keys_from_bulk(zip_url: str) -> pd.DataFrame:
    """Download one bulk ZIP and return a keys DataFrame (one row per series key)."""
    r = requests.get(zip_url, timeout=240); r.raise_for_status()
    with zipfile.ZipFile(io.BytesIO(r.content)) as z:
        csv_name = [n for n in z.namelist() if n.lower().endswith(".csv")][0]
        df = pd.read_csv(z.open(csv_name), dtype=str, low_memory=False)

    # Heuristic: dimension columns = all non-observation / non-time columns
    non_key_cols = {c for c in df.columns
                     if c.upper().startswith("OBS")
                     or any(n in c.upper() for n in ("TIME_PERIOD", "TIME_FORMAT", "UNIT_MULT")) }
    dim_cols = [c for c in df.columns if c not in non_key_cols]

    # Deduplicate to get one row per series key
    keys = (df[dim_cols].drop_duplicates().reset_index(drop=True))
    # Add flow/topic from the ZIP filename (e.g., WS_CREDIT_GAP)
    flow = csv_name.split("_csv_flat")[0].split("/)[-1]
    keys.insert(0, "flow_id", flow)
    return keys

def bis_keys(link:str) -> pd.DataFrame:
    try:
        return parse_keys_from_bulk(link)
    except Exception as e:
        print(f"[warn] {link}: {e}")

```



```

def keys_from_flow_id(flow_id: str, sess: requests.Session=SESSION, timeout: int = 240) -> pd.DataFrame:
    """Download one bulk ZIP and return a keys DataFrame (one row per series key)."""
    base = "https://data.bis.org/static/bulk"
    zip_url = f"{base}/{flow_id}_csv_flat.zip"
    sess = make_session() if sess is None else sess
    r = sess.get(zip_url, timeout=timeout)
    r.raise_for_status()
    with zipfile.ZipFile(io.BytesIO(r.content)) as z:
        csv_name = [n for n in z.namelist() if n.lower().endswith(".csv")][0]
        df = pd.read_csv(z.open(csv_name), dtype=str, low_memory=False)

    # Heuristic: dimension columns = all non-observation / non-time columns
    non_key_cols = {c for c in df.columns
                    if c.upper().startswith("OBS")
                    or any(n in c.upper() for n in ("TIME_PERIOD", "TIME_FORMAT", "UNIT_MULT")) }
    dim_cols = [c for c in df.columns if c not in non_key_cols]

    # Deduplicate to get one row per series key
    keys = (df[dim_cols].drop_duplicates().reset_index(drop=True))
    # Add flow/topic from the ZIP filename (e.g., WS_CREDIT_GAP)
    flow = csv_name.split("_csv_flat")[0].split("/)[-1]
    keys.insert(0, "flow_id", flow)
    return keys
#####

def bis_all_keys() -> pd.DataFrame:
    """Enumerate series keys for ALL BIS bulk datasets (can be large)."""
    all_links = list_bulk_csv_flat_links()
    frames = []
    for link in all_links:
        try:
            frames.append(parse_keys_from_bulk(link))
        except Exception as e:
            print(f"[warn] {link}: {e}")
    return pd.concat(frames, ignore_index=True) if frames else pd.DataFrame()

```

```

In [13]: all_links = list_bulk_csv_flat_links()
all_links

```

```
Out[13]: ['https://data.bis.org/static/bulk/WS_CBPOL_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_CBS_PUB_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_CBTA_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_CPMI_CASHLESS_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_CPMI_CT1_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_CPMI_CT2_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_CPMI_DEVICES_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_CPMI_INSTITUT_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_CPMI_MACRO_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_CPMI_PARTICIP_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_CPMI_SYSTEMS_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_CPP_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_CREDIT_GAP_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_DEBT_SEC2_PUB_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_DER_OTC_TOV_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_DPP_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_DSR_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_EER_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_GLI_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_LBS_D_PUB_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_LONG_CPI_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_NA_SEC_DSS_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_OTC_DERIV2_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_SPP_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_TC_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_XRU_csv_flat.zip',
'https://data.bis.org/static/bulk/WS_XTD_DERIV_csv_flat.zip']
```

```
In [14]: parse_keys_from_bulk(all_links[0])
```

Out[14]:

	flow_id	STRUCTURE	STRUCTURE_ID	ACTION	FREQ:Frequency	REF_AREA:Reference area	UNIT_MEASURE:Unit of measure	COMPILATION:Compi
0	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	M: Monthly	BR: Brazil	368: Per cent per year	From 5 Mar 1999 or Central Bank targ
1	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	M: Monthly	BE: Belgium	368: Per cent per year	From 1 Jan 1999 on the series is dis
2	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	M: Monthly	CL: Chile	368: Per cent per year	From 1997 onwards: c monetary polic
3	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	M: Monthly	CA: Canada	368: Per cent per year	From 1 Jun 1994 on Central Bank tai
4	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	M: Monthly	CH: Switzerland	368: Per cent per year	From 13 June 2019 or SNB Policy rate
...	...	...	...	...	...	...	...	...
93	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	D: Daily	TR: Türkiye	368: Per cent per year	From 20 May 2010 on 1 week official

	flow_id	STRUCTURE	STRUCTURE_ID	ACTION	FREQ:Frequency	REF_AREA:Reference area	UNIT_MEASURE:Unit of measure	COMPILATION:Compi
94	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	D: Daily	US: United States	368: Per cent per year	From 19 Dec 1985 on mid-point of the
95	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	D: Daily	XM: Euro area	368: Per cent per year	From 18 Sep 2024 on official centra
96	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	D: Daily	ZA: South Africa	368: Per cent per year	Official rep
97	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	M: Monthly	AU: Australia	368: Per cent per year	From 2 Aug 1990 on cash rate targe

98 rows × 12 columns

```
In [15]: bis_keys(all_links[0])
```

Out[15]:

	flow_id	STRUCTURE	STRUCTURE_ID	ACTION	FREQ:Frequency	REF_AREA:Reference area	UNIT_MEASURE:Unit of measure	COMPILATION:Compi
0	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	M: Monthly	BR: Brazil	368: Per cent per year	From 5 Mar 1999 or Central Bank targ
1	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	M: Monthly	BE: Belgium	368: Per cent per year	From 1 Jan 1999 on the series is dis
2	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	M: Monthly	CL: Chile	368: Per cent per year	From 1997 onwards: c monetary polic
3	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	M: Monthly	CA: Canada	368: Per cent per year	From 1 Jun 1994 on Central Bank tai
4	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	M: Monthly	CH: Switzerland	368: Per cent per year	From 13 June 2019 or SNB Policy rate
...	...	...	...	...	...	...	...	...
93	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	D: Daily	TR: Türkiye	368: Per cent per year	From 20 May 2010 on 1 week official

	flow_id	STRUCTURE	STRUCTURE_ID	ACTION	FREQ:Frequency	REF_AREA:Reference area	UNIT_MEASURE:Unit of measure	COMPILATION:Compi
94	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	D: Daily	US: United States	368: Per cent per year	From 19 Dec 1985 on mid-point of the
95	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	D: Daily	XM: Euro area	368: Per cent per year	From 18 Sep 2024 on official centra
96	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	D: Daily	ZA: South Africa	368: Per cent per year	Official rep
97	WS_CBPOL	dataflow	BIS:WS_CBPOL(1.0): Central bank policy rates	I	M: Monthly	AU: Australia	368: Per cent per year	From 2 Aug 1990 on cash rate targe

98 rows × 12 columns

```
In [16]: flow_id= "WS_CREDIT_GAP"
keys_from_flow_id(flow_id)
```

Out[16]:

	flow_id	STRUCTURE	STRUCTURE_ID	ACTION	FREQ:Frequency	BORROWERS_CTY:Borrowers' country	TC_BORROWERS:Borrowers' sector
0	WS_CREDIT_GAP	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	AU: Australia	P: Private non-financial
1	WS_CREDIT_GAP	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	MY: Malaysia	P: Private non-financial
2	WS_CREDIT_GAP	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	NL: Netherlands	P: Private non-financial
3	WS_CREDIT_GAP	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	NL: Netherlands	P: Private non-financial
4	WS_CREDIT_GAP	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	ZA: South Africa	P: Private non-financial
...	...	...	...	...	...	...	...
127	WS_CREDIT_GAP	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	FI: Finland	P: Private non-financial
128	WS_CREDIT_GAP	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	FI: Finland	P: Private non-financial
129	WS_CREDIT_GAP	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	IL: Israel	P: Private non-financial
130	WS_CREDIT_GAP	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	IL: Israel	P: Private non-financial
131	WS_CREDIT_GAP	dataflow	BIS:WS_CREDIT_GAP(1.0): BIS credit-to-GDP gaps	I	Q: Quarterly	IE: Ireland	P: Private non-financial

132 rows × 13 columns

```
In [ ]: # Example:  
        #df_all_keys = bis_all_keys()  
        #display(df_all_keys.head(), len(df_all_keys))
```